

บทที่ 5

การออกแบบตัวแอปพลิเคชันฝั่งเซิร์ฟเวอร์

5.1 รูปแบบโดยทั่วไปและวิธีการเล่น

รูปแบบของแอปพลิเคชันจะเป็นเกมส้อมอนไลน์บนโทรศัพท์มือถือซึ่งเขียนโดยภาษาจาวา ในรูปแบบของเครือข่ายแบบซ็อกเก็ต โดยตัวของแอปพลิเคชันนี้ได้ถูกแบ่งออกเป็น 2 ส่วนคือ ส่วนของตัวเซิร์ฟเวอร์ และ ส่วนของตัวไคลเอนต์

การเริ่มของแอปพลิเคชันนี้จะเริ่มจากตัวเซิร์ฟเวอร์เปิดให้บริการไว้ก่อน แล้วมีตัวไคลเอนต์เป็นตัวเล่น โดยเริ่มแรกตัวผู้เล่นจะต้องมีแอปพลิเคชันของเกมนี้อยู่ก่อน (โดยเป็นแอปพลิเคชันฝั่งไคลเอนต์) เมื่อต้องการจะร่วมเล่นเกมก็จะทำการเปิดแอปพลิเคชันขึ้นมาแล้วต้องทำการลงทะเบียนก่อน เมื่อลงทะเบียนแล้วก็จะสามารถร่วมเล่นเกมนี้ผ่านทางตัวเซิร์ฟเวอร์ได้

ลักษณะการเล่นของเกมเมื่อตัวไคลเอนต์ที่ได้ทำการลงทะเบียนไว้แล้ว เข้ามาร่วมเล่น ตัวเซิร์ฟเวอร์ซึ่งทำการสร้างศัตรูขึ้นมา (โดยได้ออกแบบให้มีศัตรูในฉากมีจำนวนคงที่คือ 20 ตัว) ผู้เล่นก็จะสามารถทำการฆ่าศัตรูที่มีอยู่ในฉากได้ (ศัตรูไม่สามารถทำอะไรตัวผู้เล่นได้) เมื่อทำการฆ่าศัตรู ตัวผู้เล่นก็จะมีคะแนนเพิ่มขึ้น 1 คะแนน แล้วตัวศัตรูก็จะตาย เมื่อศัตรูตายตัวเซิร์ฟเวอร์จะทำการสร้างศัตรูขึ้นมาใหม่ และทำการสร้างไอเทมขึ้นมา ณ ตำแหน่งที่ศัตรูถูกฆ่า เมื่อผู้เล่นทำการเก็บไอเทมนั้น ก็จะทำให้ผู้เล่นได้รับค่าต่างๆตามลักษณะของไอเทมนั้นๆ

เมื่อตัวไคลเอนต์ตั้งแต่สองตัวขึ้นไปออนไลน์อยู่ในระบบ ตัวไคลเอนต์แต่ละตัวก็จะสามารถมองเห็นตัวไคลเอนต์ตัวอื่นๆ ซึ่งตัวไคลเอนต์แต่ละตัวจะสามารถทำการสู้กันได้ โดยเมื่อทำการฟันแล้วตัวที่ทำการฟันจะได้คะแนนเพิ่มขึ้นมา 1 คะแนน ส่วนตัวที่ถูกฟันจะมีพลังชีวิตลดลงตามระดับความสามารถของตัวผู้เล่นนั้นๆ

เมื่อตัวไคลเอนต์ได้ทำคะแนนถึงจุดแต่ละจุดก็จะเป็นการเพิ่มระดับความสามารถของตัวผู้เล่นขึ้น คือเมื่อมีคะแนนถึง 0 - 4 คะแนนก็จะทำให้มีในระดับความสามารถเท่ากับ 1 คะแนน 5 - 9 คะแนนจะมีในระดับความสามารถเท่ากับ 2 คะแนน 10 - 14 คะแนนจะมีในระดับความสามารถเท่ากับ 3 เป็นต้นไปเรื่อยๆ โดยความแตกต่างระหว่างระดับความสามารถจะแตกต่างกัน คือเมื่อตัวผู้เล่นโดนฟันพลังชีวิตที่ลดลงจะแตกต่างกัน คือเมื่อผู้เล่นที่มีระดับอยู่ในระดับความสามารถ 3 ขึ้นไป เมื่อโดนฟัน พลังชีวิตจะลดลง 1 หน่วย ถ้าอยู่ในระดับความสามารถ 2 เมื่อโดนฟัน พลังชีวิตจะลดลง 3 หน่วยและถ้าอยู่ในระดับ 1 เมื่อโดนฟัน พลังชีวิตจะลดลง 5 หน่วย และความแตกต่างระหว่างระดับความสามารถอีกอย่างคือเมื่อมีความสามารถเพิ่มขึ้นที่เก็บพลังชีวิตจะมีเพิ่มขึ้นกล่าวคือเมื่อแรกเริ่มที่เก็บพลังชีวิตจะอยู่ที่ 20 หน่วยเมื่อเพิ่ม 1 ระดับความสามารถก็จะทำให้เพิ่มที่เก็บพลังชีวิตเพิ่มขึ้นระดับละ 2 หน่วย

เมื่อตัวผู้เล่นตาย ก็จะมีการเก็บคะแนนของผู้เล่นคนนั้นไว้ แล้วทำการตัวผู้เล่นออกจากระบบ เพื่อให้ไคลเอนต์ตัวอื่นๆ ไม่สามารถมองเห็นตัวที่ตายไปแล้ว แล้วเมื่อผู้เล่นนั้นเข้ามาใหม่ก็จะทำการเอาข้อมูลที่เก็บเอาไว้มาใช้ คือเอาคะแนนเดิมก่อนตายมาใช้เมื่อเริ่มใหม่ โดยที่การเริ่มใหม่ตัวผู้เล่นจะมีพลังชีวิตเต็มที่เก็บพลังชีวิตของตัวเอง

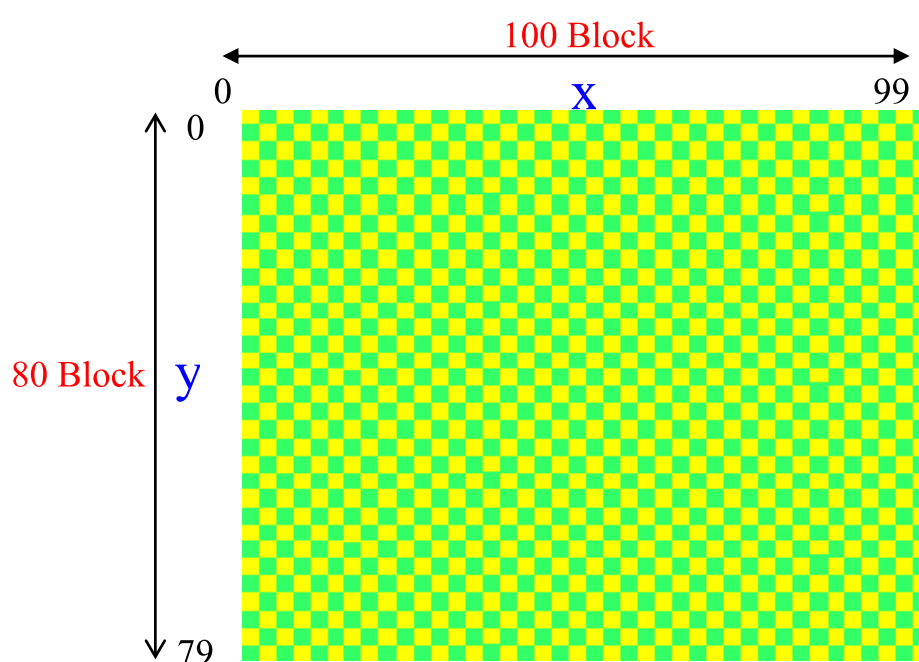
เมื่อผู้เล่นออกจากระบบก็จะทำการเก็บคะแนนของผู้เล่นคนนั้นไว้เช่นกัน แล้วจะทำการตัดผู้เล่นคนนั้นออกจากระบบเพื่อให้ไคลเอนต์ตัวอื่นๆ ไม่สามารถมองเห็นตัวที่ออกจากระบบไปแล้ว

5.2 ข้อมูลโดยทั่วไปของแอปพลิเคชัน

ในตัว เซิร์ฟเวอร์ จะมี Record Store อยู่ 2 ตัวด้วยกัน คือ Record Store ตัวแรกจะทำการเก็บค่าการลงทำเบียนของ ไคลเอนต์ (เก็บ IP Port ID และคะแนน) ส่วนตัวที่สองจะเก็บค่าตัวที่กำลังออนไลน์อยู่ (เก็บ IP Port และ ID)

ในตัว ไคลเอนต์ ก็จะไม่มีการเก็บค่า IP Port และ ID ที่ได้จาก เซิร์ฟเวอร์ ในตอนลงทะเบียนไว้ที่ตัวแปรในส่วนของตัวเอง ไคลเอนต์

การเก็บค่าตำแหน่งต่างๆของแผนที่จะถูกเก็บไว้ในค่า Map ซึ่งเป็นอาร์เรย์ 2 มิติ ของ Character โดยเป็นอาร์เรย์ที่มีขนาด 100×80 ซึ่งในแต่ละ $\text{Map}[x][y]$ จะถูกเรียกว่าเป็น Block ดังนั้นใน Map ทั้งหมด จะมีทั้งหมด 8000 Block



รูปที่ 5.1 การจำลองแผนที่

ในแต่ละ Block จะเก็บค่าที่เป็น Character โดย Character แต่ละตัวจะมีความหมายที่แตกต่างกัน ดังตาราง

Char	ความหมาย	Char	ความหมาย
a	ผู้เล่นหันหน้า	j	ผู้เล่นหันซ้าย
b	ผู้เล่นหันหน้ากำลังเดิน	k	ผู้เล่นหันซ้ายกำลังเดิน
c	ผู้เล่นหันหน้ากำลังฟัน	l	ผู้เล่นหันซ้ายกำลังฟัน
d	ผู้เล่นหันหลัง	m	กำแพง
e	ผู้เล่นหันหลังกำลังเดิน	n	ก้อนหิน
f	ผู้เล่นหันหลังกำลังฟัน	o	หญ้า
g	ผู้เล่นหันขวา	t,u	Item
h	ผู้เล่นหันขวากำลังเดิน	z	ศัตรู
i	ผู้เล่นหันขวากำลังฟัน		

ตารางที่ 5.1 แสดงความหมายของ character แต่ละตัว

เช่น $\text{Map}[x][y] = 'a'$ หมายความว่าที่ตำแหน่งแกนนอน x และแกนตั้ง y มีผู้เล่นที่หันหน้าอยู่ในตำแหน่งนี้ หรือ $\text{Map}[x][y] = 'o'$ หมายความว่าที่ตำแหน่งแกนนอน x และแกนตั้ง y เป็นตำแหน่งหญ้า

การแสดงผลที่หน้าจอจะเป็นการเอาค่าที่อยู่ใน Map ที่เราต้องการ ซึ่งเป็น Character มาเปรียบเทียบกับค่าอะไร ถ้าเป็นค่าไหนก็ให้แสดงรูปนั้น เช่น ถ้าเป็น 'a' ก็จะแสดงรูปผู้เล่นหันหน้า ถ้าเป็น 'm' ก็จะแสดงรูปกำแพง

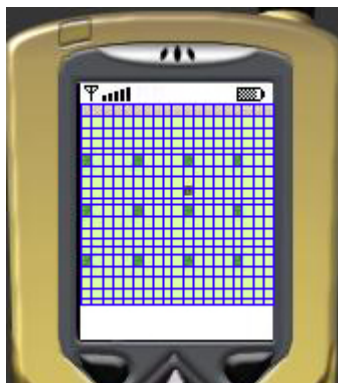
โดยการแสดงที่หน้าจอได้ทำให้มีการแสดงเป็น 2 มุมมอง

1. ค่า zoom มีค่าเป็น 0 จะได้มุมมองที่แสดงเป็น 5 x 5 Block โดยการแสดงรูปจะใช้รูปที่มีขนาด 19 x 20 pixels



รูปที่ 5.2 หน้าจอในมุมมองแบบ 5 x 5

2. ค่า zoom มีค่าเป็น 1 จะได้มุมมองที่แสดงเป็น 19 x 20 Block โดยการแสดงรูปจะใช้รูปที่มีขนาด 5 x 5 pixels



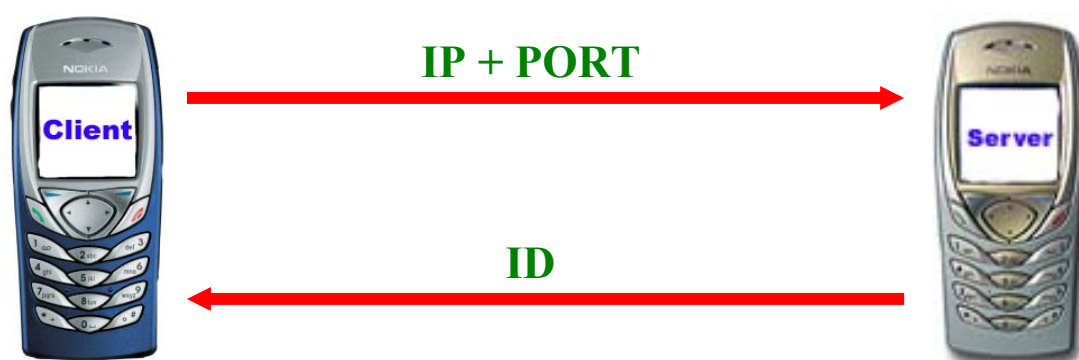
รูปที่ 5.3 หน้าจอในมุมมองแบบ 19 x 20

5.3 การทำงานฝั่งเซิร์ฟเวอร์และฟังก์ชันต่างๆ

การทำงานในฝั่งเซิร์ฟเวอร์ได้มีการทำงานหลายอย่างซึ่งเป็นการทำงานที่ต้องรับข้อมูลมาจากไคลเอนต์มาทำตามฟังก์ชันต่างๆตามที่ได้รับข้อมูลมา แล้วทำการส่งต่อไปยังไคลเอนต์ที่ต้องส่งตามฟังก์ชัน โดยในตัวเซิร์ฟเวอร์เองจะมีหลายการทำงานแบ่งออกเป็น

5.3.1 การลงทะเบียน

เมื่อเซิร์ฟเวอร์ได้รับแพ็กเกจเข้ามาก็จะมาทำการตรวจสอบว่าเป็นแพ็กเกจชนิดไหน โดยจะตรวจสอบจากความยาวของแพ็กเกจ เมื่อทราบแล้วว่าเป็นแพ็กเกจของการลงทะเบียน เซิร์ฟเวอร์ก็จะทำการสร้างค่า ID ขึ้นมาแล้วก็จะทำการบันทึกค่า IP Port (ไคลเอนต์เป็นตัวส่งมา) ID และบันทึกค่าคะแนนเริ่มต้นให้กับ ID นั้นๆ แล้วเซิร์ฟเวอร์ก็จะทำการส่งค่า ID ไปให้กับ ไคลเอนต์



รูปที่ 5.4 การลงทะเบียน

5.3.2 การสร้างศัตรูขึ้นมา

เมื่อเซิร์ฟเวอร์เปิดขึ้นก็จะมีการสร้างศัตรูขึ้นมาโดยการออกแบบของแอปพลิเคชันนี้ได้ ออกแบบไว้ว่าต้องมีศัตรูคงที่ในแผนที่ คือ 20 ตัว โดยเริ่มต้นจะสร้างศัตรูด้วย `gen_perm()` โดยในฟังก์ชันนี้จะมีการเรียก `generateRandoms()` `keep_mon()` และ `table()` โดยใน `table()` จะมีการเรียกอีก 1 ฟังก์ชัน คือ `generateRandoms_die()` โดยเมื่อสร้างศัตรูได้แล้วซึ่งเก็บค่าไว้แล้วในตัวแปร `pos_x[ ]` และ `pos_y[ ]` โดยการสร้างศัตรูขึ้นมานี้จะมีการตรวจสอบว่าในการสร้างนี้ตัวศัตรูที่สร้างขึ้นจะไม่ชนกับตำแหน่งของ กำแพง ก้อนหิน

`gen_perm()` เป็นฟังก์ชันที่ใช้ในการสร้างศัตรูขึ้นมาโดยฟังก์ชันนี้จะถูกเรียกเมื่อตัวเซิร์ฟเวอร์ได้ถูกเปิดขึ้นมา โดยที่ตัว `gen_perm()` นี้จะมีการเรียกฟังก์ชันต่างๆเพื่อใช้ในการเก็บตำแหน่งของศัตรูและตรวจสอบการทับกันของตำแหน่ง

`generateRandoms()` เป็นฟังก์ชันที่ทำการ random ค่าตำแหน่งของศัตรูขึ้นมาทั้ง 20 ตำแหน่งโดยเมื่อ random ขึ้นมาแล้วจะถูกเก็บไว้เป็นสตริงที่มีความยาว 80 (ศัตรู 1 ตัวใช้ 4 ตำแหน่งเพื่อบอกตำแหน่งในตาราง)

`keep_mon()` เป็นฟังก์ชันที่มีไว้เพื่อทำการเก็บค่าตำแหน่งของศัตรูเก็บไว้ในตัวแปรอาร์เรย์ คือเก็บไว้ใน `pos_x[ ]` คือตำแหน่ง x ของศัตรู และ `pos_y[ ]` คือตำแหน่ง y ของศัตรู โดยในการเก็บค่าเหล่านี้ จะนำเอาค่าสตริงที่ได้จาก `generateRandoms()` มาทำการตัด คือ ใน 2 ตำแหน่งแรกของสตริงก็จะให้เก็บไว้ใน `pos_x[0]` และ 2 ตำแหน่งถัดไปก็จะให้เก็บไว้ใน `pos_y[0]` และ 2 ตำแหน่งถัดไปก็จะเก็บไว้ใน `pos_x[1]` และ 2 ตำแหน่งถัดไปก็จะให้เก็บไว้ใน `pos_y[1]` ทำเช่นนี้ไปเรื่อยๆจนสิ้นสุดของสตริงก็จะได้ตำแหน่งของศัตรูทั้งหมด 20 ตัว เก็บไว้ในอาร์เรย์ของ `int`

`table()` เป็นฟังก์ชันที่ใช้สำหรับตรวจสอบว่าตำแหน่งศัตรูที่สร้างขึ้นมาไปทับกับตำแหน่งอื่นหรือไม่ เช่น ทับกับตำแหน่งกำแพง ทับกับตำแหน่งก้อนหิน ถ้าทับกันก็จะให้เรียก `generateRandoms_die()` เพื่อทำการสร้างตำแหน่งขึ้นมาใหม่แทนที่ศัตรูตัวนั้น โดยการตรวจสอบจะใช้ `while(Map[pos_x[k]][pos_y[k]] != 'o')` โดย 'o' คือตำแหน่งหญ้า (สามารถให้เกิดศัตรูได้) ถ้าเข้ากรณีนี้จะทำการให้ สร้างตำแหน่งขึ้นมาใหม่แล้วทำการเก็บไว้ใน `pos_x[k]` และ `pos_y[k]` แล้วถ้าตำแหน่งที่สร้างใหม่ขึ้นมาแล้วยังมีค่าเท่ากันอยู่ก็จะต้องทำการสร้างขึ้นมาใหม่

```
while(Map[pos_x[k]][pos_y[k]] != 'o')
{
    String new_mon = generateRandoms_die();
    String one = new_mon.substring(0,2);
    pos_x[k] = Integer.parseInt(one);
    String two = new_mon.substring(2,4);
    pos_y[k] = Integer.parseInt(two);
}
```

`generateRandoms_die()` เป็นฟังก์ชันที่ใช้ในการ `random` ค่าสุ่มขึ้นมาใหม่โดยการ `random` ค่าสุ่มนั้นจะทำการ `random` เพียงค่า `x` กับ `y` เท่านั้น คือตำแหน่งของศัตรูเพียงตัวเดียว

5.3.3 การเริ่มเล่นของไคลเอนต์

เมื่อเซิร์ฟเวอร์ได้รับแพ็กเกจจากไคลเอนต์แล้วได้รับการตรวจสอบแล้วว่าเป็นแพ็กเกจของการเริ่มต้น เซิร์ฟเวอร์ก็จะทำการสร้างตำแหน่งการเกิดให้กับตัวไคลเอนต์แล้วส่งไปพร้อมกับค่าของศัตรูที่ได้ทำการสร้างขึ้น ในการส่งครั้งนี้จะมีการเอาค่าของคะแนนที่ได้เก็บไว้ใน `Record Store` มารวมส่งไปให้กับตัวไคลเอนต์ด้วย เมื่อส่งไปแล้วตัวเซิร์ฟเวอร์ต้องทำการส่งค่าตำแหน่งของตัวที่เข้ามาใหม่ไปให้กับตัวไคลเอนต์อื่นๆที่ออนไลน์อยู่ให้รู้ตำแหน่งของไคลเอนต์ตัวใหม่เพื่อที่จะได้ทำการเปลี่ยนค่าใน `Map` ให้มีค่าของไคลเอนต์ตัวใหม่ เซิร์ฟเวอร์ยังต้องส่งค่าตำแหน่งของไคลเอนต์ตัวอื่นๆ ไปให้กับตัวไคลเอนต์ที่เข้ามาใหม่เพื่อทำการแสดงตำแหน่งของไคลเอนต์ตัวอื่นๆที่ออนไลน์อยู่ด้วย

โดยการส่งตำแหน่งไคลเอนต์ตัวอื่นๆ ไปให้กับไคลเอนต์ตัวใหม่จะใช้ `sendfirst()` และการส่งค่าตำแหน่งของไคลเอนต์ตัวใหม่ไปให้กับไคลเอนต์ตัวอื่นๆจะใช้ `sendboard()`

`sendfirst()` เป็นฟังก์ชันที่จะส่งไปให้กับไคลเอนต์ตัวใหม่ที่เพิ่งเข้ามา โดยจะเริ่มจากการเปิด `Record Store` เพื่อดูว่ามีไคลเอนต์ตัวอื่นๆที่ออนไลน์อยู่หรือไม่ ถ้ามีก็จะทำการเอาค่า `ID` นั้นมาจากตารางเพื่อเอาค่าตำแหน่งของ `ID` นั้นจาก `point[id]` มาเพื่อส่งไปที่ตัวไคลเอนต์ตัวใหม่โดยในการส่งไปที่ไคลเอนต์ตัวใหม่นี้ก็จะไปดึงค่า `IP` และ `Port` จาก `Record Store` ของการลงทะเบียนเพื่อจะได้ส่งไปให้กับไคลเอนต์ตัวที่เข้ามาใหม่ได้อย่างถูกต้อง ถ้ามีไคลเอนต์ตัวอื่นๆมากกว่า 2 ตัวที่ต้องวนลูปเพื่อทำการส่ง การส่งก็จะทำการส่งเหมือนกับ `ID` แรกที่ส่ง

โดยการดึงค่าออกมาจาก `Record` จะใช้

```
len = rs.getRecord(i,recData,0);
```

```
rec = new String(recData,0,len);
```

เมื่อทำแล้วจะได้ `Record` ในตำแหน่งที่ `i` ออกมาเพื่อมาทำการตรวจสอบและจะส่งด้วย

```
send(IP_new_client, position_another_client)
```

`sendboard()` เป็นฟังก์ชันที่ใช้สำหรับส่งตำแหน่งของไคลเอนต์ตัวใหม่ไปให้กับไคลเอนต์ตัวอื่นๆที่ออนไลน์อยู่ โดยเริ่มจากการเปิด `Record Store` เพื่อดูว่ามีไคลเอนต์ตัวอื่นๆที่ออนไลน์อยู่หรือไม่ ถ้ามีก็จะทำการเอาค่า `ID` เหล่านั้นมาใช้เพื่อไปดึงค่า `IP` และ `Port` ของแต่ละ `ID` เพื่อที่จะทำการส่ง ถ้ามีไคลเอนต์ตัวอื่นๆมากกว่า 2 ตัวก็จะทำการวนลูปเพื่อทำการส่ง

ในการส่งนี้ใช้

```
send(IP_anther_client, position_new_client);
```

5.3.4 การเดินและการหันของไคลเอนต์

เมื่อเซิร์ฟเวอร์ได้รับแพ็กเกจจากไคลเอนต์ แล้วได้รับการตรวจสอบแล้วว่าเป็นแพ็กเกจของการเดินและการหัน เซิร์ฟเวอร์ก็ต้องทำการตัดค่าเหล่านี้เพื่อจะเอาค่า `ID` ที่ส่งมาไปทำการเก็บตำแหน่งของตัวไคลเอนต์ เมื่อทำการเก็บตำแหน่งแล้วก็จะต้องทำการส่งค่าตำแหน่งนี้ไปให้กับไคลเอนต์ตัวอื่นๆ

```

if (stringpoint.length()==8) {
    StringToInt();
    keepoint();
    sendboard();
}

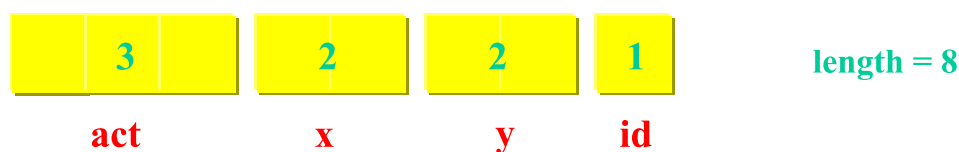
```

เมื่อ `stringpoint` คือ ค่าที่รับมาจากไคลเอนต์ซึ่งถ้ามีความยาวเท่ากับ 8 แสดงว่าเป็นแพ็กเกจของการเดินและการหันเมื่อเข้ามาแล้วจะไปเข้า `StringToInt()` เพื่อทำการแปลงค่าเป็น `int` เสร็จแล้วก็จะไปเข้า `keepoint()` เพื่อทำการเก็บตำแหน่ง เสร็จแล้วก็จะไปเข้า `sendboard()` เพื่อทำการส่งค่าตำแหน่งไปให้กับตัวอื่นๆ

`StringToInt()` เป็นฟังก์ชันที่การเอาค่าที่ได้รับเข้ามา มาทำการดึงเอาค่า ID ของ ไคลเอนต์ เพื่อมาทำการเก็บไว้ในตัวแปร `idstr`

`keepoint()` เป็นฟังก์ชันที่เก็บค่าที่รับเข้ามาไว้ใน `point[idstr]`

`sendboard()` เป็นฟังก์ชันเดียวกันกับในหัวข้อ 5.3.3 คือจะส่งค่าตำแหน่ง(แพ็กเกจที่มีความยาวเท่ากับ 8)ไปให้กับไคลเอนต์ตัวอื่นๆ ที่ออนไลน์อยู่



รูปที่ 5.5 แพ็กเกจในกรณีการเดินและการหันของเซิร์ฟเวอร์ที่จะส่ง

act	หมายถึง ค่าที่แสดงการกระทำของตัวผู้เล่น
x	หมายถึง ค่าตำแหน่ง x ของตัวผู้เล่น
y	หมายถึง ค่าตำแหน่ง y ของตัวผู้เล่น
id	หมายถึง ค่า id ของตัวผู้เล่น

5.3.5 การฆ่าศัตรู

เมื่อเซิร์ฟเวอร์ได้รับแพ็กเกจจากไคลเอนต์ แล้วได้รับการตรวจสอบแล้วว่าเป็นแพ็กเกจของการฆ่าศัตรู เซิร์ฟเวอร์ก็จะเข้าฟังก์ชันของการฆ่าศัตรู

```

if(stringpoint.length()==9){
    StringToInt();
    if (stringpoint.substring(0,1).compareTo("1") == 0)
        ManageKill();
}

```

เมื่อ stringpoint คือ ค่าที่รับมาจากไคลเอนต์ซึ่งถ้ามีความยาวเท่ากับ 9 แล้วจะเข้าฟังก์ชัน StringToInt() เพื่อทำการแปลงค่าเป็น int แล้วก็จะไปทำการตรวจสอบในตำแหน่งแรกว่ามีค่าเท่ากับ “1” หรือไม่ ถ้าเท่าแสดงว่าเป็นแพ็กเกจของการฆ่าศัตรู ก็จะไปเข้า ManageKill()

Stringpoint() เป็นฟังก์ชันเดียวกันกับในหัวข้อ 5.3.4 ซึ่งจะเก็บ ID ตัวที่ฆ่าศัตรูไว้ที่ตัวแปร idstr

ManageKill() เป็นฟังก์ชันที่จัดการกับการฆ่าศัตรูของไคลเอนต์ เมื่อเข้าฟังก์ชันนี้แล้วก็จะทำการตัดค่าต่างๆที่รับมาเก็บไว้ แล้วทำการหาตำแหน่งของศัตรูที่ต้องการฆ่า โดยจะดูจากตำแหน่งของไคลเอนต์จากที่รับมา แล้วดูว่าหันหน้ามาไปทางใด จะรู้ตำแหน่งศัตรูที่ต้องการฆ่า เมื่อรู้ตำแหน่งแล้ว ก็จะทำการตรวจสอบว่าตำแหน่งที่ต้องการฆ่านั้นมีศัตรูอยู่จริงหรือไม่ ถ้ามีอยู่จริงแสดงว่าสามารถฆ่าได้ ก็จะทำการเปลี่ยนตำแหน่งนั้นให้เป็น Item แล้วก็ทำการสร้างศัตรูขึ้นมาใหม่ แล้วก็ทำการส่งแพ็กเกจที่มีความยาวเท่ากับ 12 ไปให้กับไคลเอนต์ทุกๆตัวเพื่อทำการแก้ไข Map

ทำการตรวจสอบว่าค่าตำแหน่งศัตรูที่ต้องการฟันมีอยู่จริงหรือไม่ด้วย

```
if((pos_x[j] == mon_kill_x)&&(pos_y[j] == mon_kill_y))
```

ทำการเปลี่ยนค่าให้เป็น Item ด้วย

```
Map[mon_kill_x][mon_kill_y] = 't';
```

ทำการส่งให้ไปให้ไคลเอนต์ทุกๆตัวด้วย

```
send(IP_all_client, string_managekill);
```



รูปที่ 5.6 แพ็กเกจในกรณีการฆ่าศัตรูของเซิร์ฟเวอร์ที่จะส่ง

new_monster_position	หมายถึง ค่าตำแหน่ง x และ y ของศัตรูตัวใหม่ที่ได้ random ขึ้นมา
id_mon	หมายถึง ค่า id ของศัตรูตัวที่ random ขึ้นมาใหม่
Item_position	หมายถึง ค่าตำแหน่ง x และ y ของ Item ที่ได้สร้างขึ้นมา
id_item	หมายถึง รหัสของ Item
id_killer	หมายถึง ค่า id ของตัวที่ฆ่าศัตรูได้

5.3.6 การเก็บ Item

เมื่อเซิร์ฟเวอร์ได้รับแพ็กเกจจากไคลเอนต์ แล้วได้รับการตรวจสอบแล้วว่าเป็นแพ็กเกจของการฆ่าเก็บ Item เซิร์ฟเวอร์ก็จะเข้าฟังก์ชันของการเก็บ Item

```
if(stringpoint.length()==9){
    StringToInt();
```



```
if (stringpoint.substring(0,1).compareTo("2") == 0)
```

```
    ManageItem();
```

```
}
```

เมื่อ stringpoint คือ ค่าที่รับมาจากไคลเอนต์ซึ่งถ้ามีความยาวเท่ากับ 9 แล้วจะเข้าฟังก์ชัน StringToInt() เพื่อทำการแปลงค่าเป็น int แล้วก็จะไปทำการตรวจสอบในตำแหน่งแรกว่ามีค่าเท่ากับ “2” หรือไม่ ถ้าเท่าแสดงว่าเป็นแพ็กเกจของการเก็บ Item ก็จะไปเข้า ManageItem();

Stringpoint() เป็นฟังก์ชันเดียวกันกับในหัวข้อ 5.3.4 ซึ่งจะเก็บ ID ไคลเอนต์ตัวที่ทำการเก็บ Item ไว้ที่ตัวแปร idstr

ManageItem() เป็นฟังก์ชันที่จัดการกับการเก็บ Item ของไคลเอนต์ เมื่อเข้าฟังก์ชันนี้แล้ว ก็จะทำการตัดค่าต่างๆที่รับมาเก็บไว้ แล้วทำการหาตำแหน่งของ Item ที่ต้องการเก็บ โดยจะดูจากตำแหน่งของไคลเอนต์จากที่รับมา แล้วดูว่าหันหน้าเดินไปทางใด จะได้รู้ตำแหน่ง Item ที่ต้องการเก็บ เมื่อรู้ตำแหน่งแล้ว ก็จะทำการตรวจสอบดูว่าตำแหน่งที่ต้องการเก็บนั้นมี Item อยู่จริงหรือไม่ ถ้ามีอยู่จริงแสดงว่าสามารถเก็บ Item นั้นได้ ก็จะทำการเปลี่ยนตำแหน่งนั้นให้ไปเป็นหญ้าเหมือนเดิม แล้วก็ทำการส่งแพ็กเกจที่มีความยาวเท่ากับ 7 ไปให้กับไคลเอนต์ทุกๆตัวเพื่อทำการแก้ไข Map

ทำการตรวจสอบว่าค่าตำแหน่ง Item ที่ต้องการเก็บว่ามีอยู่จริงหรือไม่ด้วย

```
if(item[j] == keep_item_xy)
```

ทำการเปลี่ยนค่าให้เป็น Item ด้วย

```
Map[keep_item_x][keep_item_y] = 'o';
```

ทำการส่งให้ไปให้ไคลเอนต์ทุกๆตัวด้วย

```
send(IP_all_client, string_manageitem);
```



รูปที่ 5.7 แพ็กเกจในกรณีการเก็บ Item ของเซิร์ฟเวอร์ที่จะส่ง

x_item หมายถึง ค่าตำแหน่ง x ของ Item

y_item หมายถึง ค่าตำแหน่ง y ของ Item

id_item หมายถึง ค่า id ของ Item

id_keep_item หมายถึง ค่า id ของผู้เล่นที่เก็บ Item

5.3.7 การสู้กับผู้เล่นคนอื่น

เมื่อเซิร์ฟเวอร์ได้รับแพ็กเกจจากไคลเอนต์ แล้วได้รับการตรวจสอบแล้วว่าเป็นแพ็กเกจของการสู้กับผู้เล่นคนอื่น เซิร์ฟเวอร์ก็จะเข้าฟังก์ชันของการสู้กับผู้เล่นคนอื่น

```
if(stringpoint.length()==9){
```

```
StringToInt();
if (stringpoint.substring(0,1).compareTo("3") == 0)
    ManageKillActor();
}
```

เมื่อ stringpoint คือ ค่าที่รับมาจากไคลเอนต์ซึ่งถ้ามีความยาวเท่ากับ 9 แล้วจะเข้าฟังก์ชัน StringToInt() เพื่อทำการแปลงค่าเป็น int แล้วก็จะไปทำการตรวจสอบในตำแหน่งแรกว่ามีค่าเท่ากับ “3” หรือไม่ ถ้าเท่าแสดงว่าเป็นแพ็คเกจของการสู้กับผู้เล่นคนอื่นก็จะไปเข้า ManageKillActor();

Stringpoint() เป็นฟังก์ชันเดียวกันกับในหัวข้อ 5.3.4 ซึ่งจะเก็บ ID ไคลเอนต์ตัวที่ทำการสู้กับผู้เล่นคนอื่นไว้ที่ตัวแปร idstr

ManageItem() เป็นฟังก์ชันที่จัดการกับการสู้กับผู้เล่น ของไคลเอนต์ เมื่อเข้าฟังก์ชันนี้แล้ว ก็จะทำการตัดค่าต่างๆที่รับมาเก็บไว้ แล้วทำการหาตำแหน่งของผู้เล่นคนอื่นที่ต้องการสู้ โดยจะดูจากตำแหน่งของไคลเอนต์จากที่รับมา แล้วดูว่าหันหน้าฟินไปทางใด จะได้รู้ตำแหน่งของผู้เล่นคนอื่นที่ต้องการสู้ เมื่อรู้ตำแหน่งแล้ว ก็จะทำการตรวจสอบว่าตำแหน่งที่ต้องการสู้นั้นมีผู้เล่นคนอื่น อยู่จริงหรือไม่ ถ้ามีอยู่จริงแสดงว่าสามารถฟินผู้เล่นผู้นั้นได้ แล้วก็ทำการส่งแพ็คเกจที่มีความยาวเท่ากับ 6 ไปให้กับไคลเอนต์ตัวที่ทำการสู้กับผู้เล่นตัวที่ถูกฟิน

ทำการตรวจสอบว่าค่าตำแหน่ง Item ที่ต้องการเก็บว่ามีอยู่จริงหรือไม่ด้วย

```
if(point[j].substring(3,7).compareTo(stringkill_actor) == 0)
    ทำการส่งให้ไปให้ไคลเอนต์ตัวที่ทำการต่อสู้กับไคลเอนต์ตัวที่ถูกฟิน
    send(IP_both_client, string_managekillactor);
```



รูปที่ 5.8 แพ็คเกจในการสู้กับผู้เล่นคนอื่นของเซิร์ฟเวอร์ที่จะส่ง

- x หมายถึง ค่าตำแหน่ง x ของตัวผู้เล่นที่โดนฟิน
- y หมายถึง ค่าตำแหน่ง y ของตัวผู้เล่นที่โดนฟิน
- id หมายถึง ค่า id ของตัวที่โดนฟิน
- id_killer หมายถึง ค่า id ของตัวที่ทำการฟิน

5.3.8 การออกจากการเล่น

เมื่อเซิร์ฟเวอร์ได้รับแพ็คเกจเข้ามา เมื่อตรวจสอบแล้วว่าเป็นแพ็คเกจของการออกจากการเล่น เซิร์ฟเวอร์ก็จะทำการตัดค่า ID และคะแนนที่ไคลเอนต์ส่งมาแล้วทำการมาเก็บไว้ใน Record Store ของการลงทะเบียนโดยดึงค่าจาก Record Store ที่มี ID เดียวกันกับที่ไคลเอนต์ส่งมา แล้ว

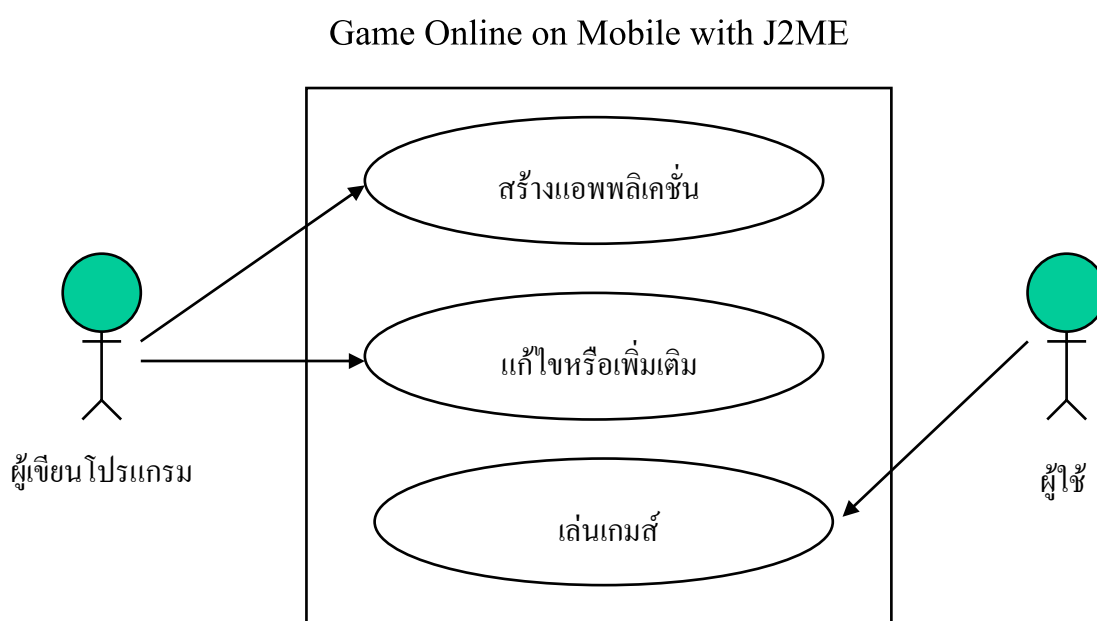
ทำการบันทึกใหม่ โดยมีค่าคะแนนที่เปลี่ยนไป เมื่อทำการบันทึกแล้วตัวเซิร์ฟเวอร์ก็จะทำการส่งค่าตำแหน่งที่เก็บไว้ใน point[id] ไปให้กับไคลเอนต์ตัวอื่นๆที่ยังออนไลน์อยู่

ทำการส่งให้ไปให้ไคลเอนต์ตัวอื่นๆที่ยังออนไลน์อยู่ด้วย

send(IP_client_out, positon_client_out)

5.4 ฟีเจอร์ต่างๆของตัวแอปพลิเคชัน

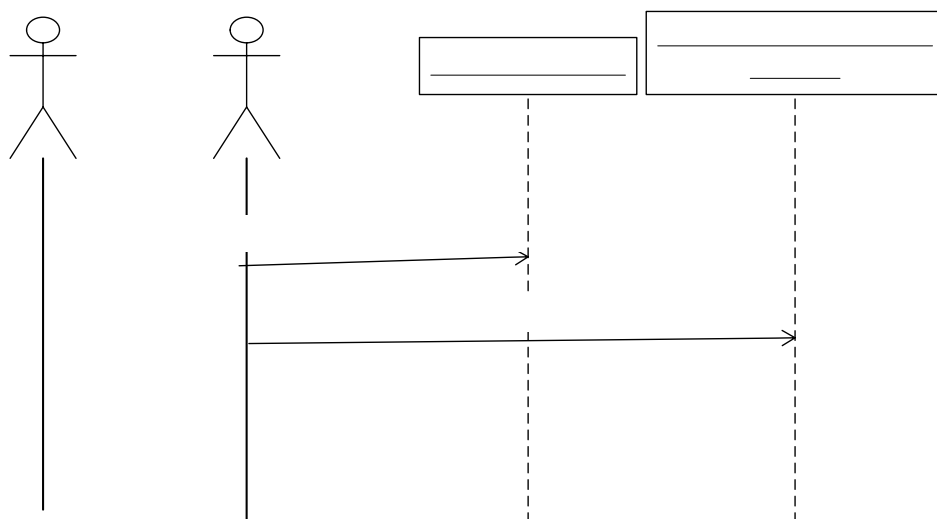
5.4.1 Use Case Diagram



รูปที่ 5.9 Use Case Diagram

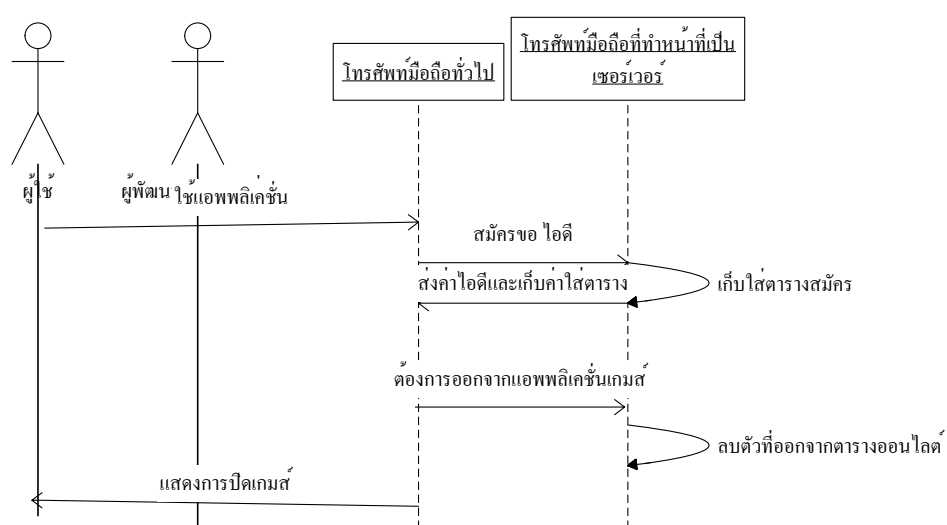
5.4.2 Sequence Diagram

5.4.2.1 การพัฒนาแอปพลิเคชัน



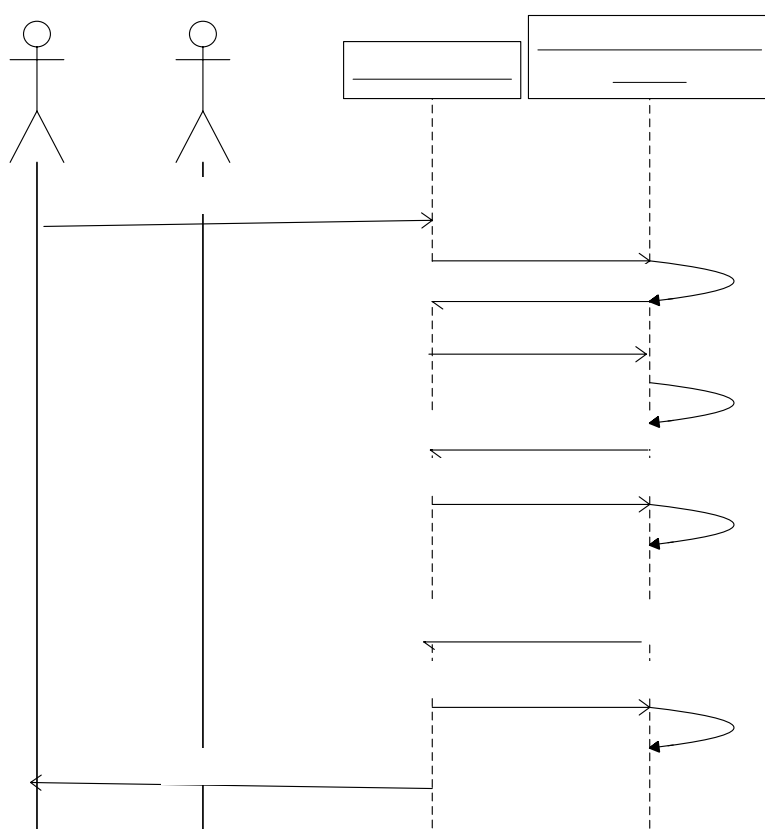
รูปที่ 5.10 *Sequence Diagram* ของการพัฒนาแอปพลิเคชัน

5.4.2.2 การลงทะเบียน

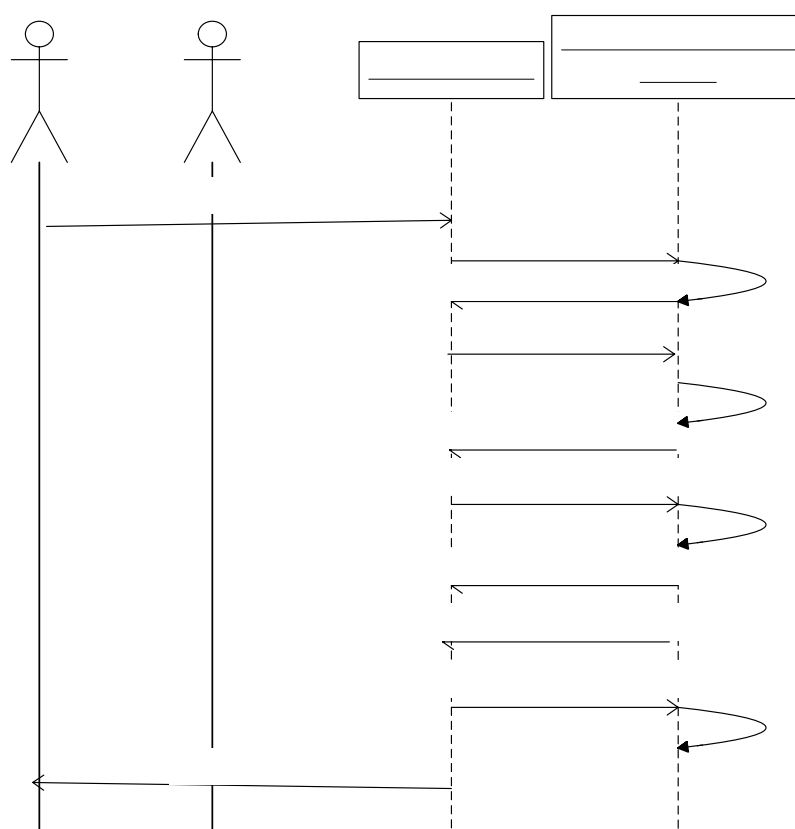


รูปที่ 5.11 Sequence Diagram ของการลงทะเบียน

5.4.2.3 การฆ่าศัตรู

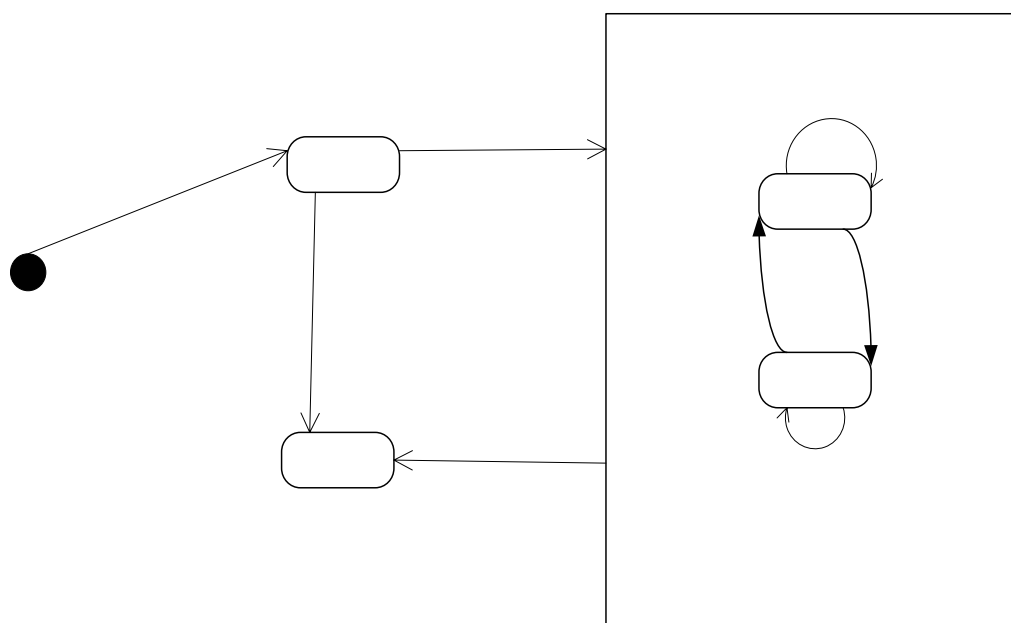
รูปที่ 5.12 *Sequence Diagram* ของการฆ่าศัตรู

5.4.2.4 การเก็บ Item



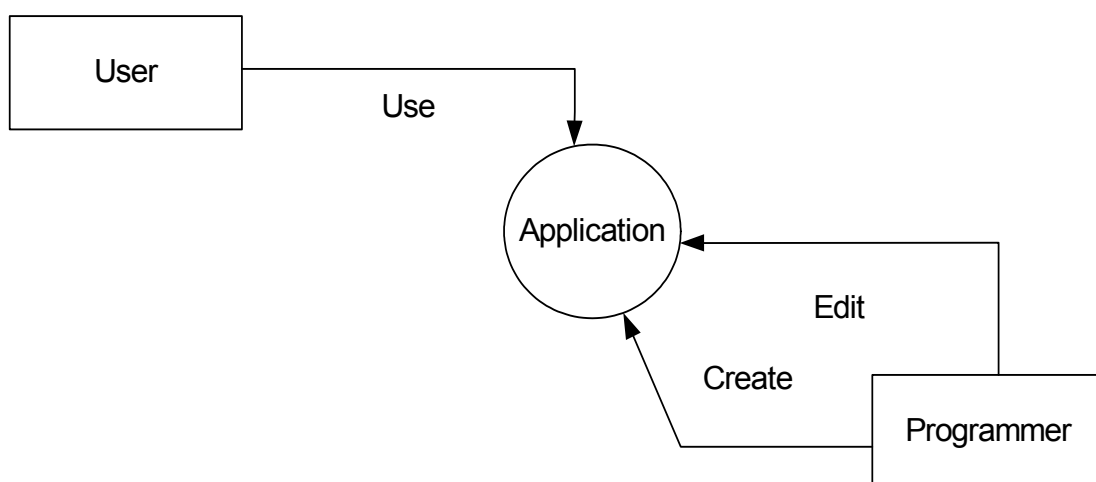
รูปที่ 5.13 Sequence Diagram ของการเก็บ Item

5.4.3 State Chart Diagram



รูปที่ 5.15 *State Chart Diagram*

5.4.4 Context Diagram



รูปที่ 5.16 Context Diagram